

[7] SingleStore-V: An Integrated Vector Database System in SingleStore

저자: Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu 외 다수

학회/년도: VLDB 2024 (Proceedings of the VLDB Endowment, Vol. 17, No. 12)

분량: 약 14페이지

역할군: (C) 경쟁/비교 시스템

요약

SingleStore-V는 기존의 분산 관계형 데이터베이스인 SingleStore에 벡터 검색 기능을 통합한 시스템이다. 범용 벡터 데이터베이스에 해당하며, SQL 쿼리 내에서 벡터 검색과 관계형 분석을 동시에 수행할 수 있다.

이 논문의 핵심 기술 기여는 **Top() 물리적 연산자**로, ORDER BY + LIMIT 패턴을 인식하여 테이블 스캔 단계에서 직접 top-k를 찾는 방식이다. 또한 **세그먼트 단위 인덱스 구조**를 통해 분산 환경에서의 확장성을 확보하고, **플러그형 벡터 인덱스**로 Faiss, hnswlib 등 다양한 인덱스를 동적으로 선택할 수 있도록 설계했다.

핵심 기술:

1. **Top() 연산자:** ORDER BY + LIMIT의 효율적 처리
2. **세그먼트 기반 인덱스:** 불변 세그먼트마다 독립적 인덱스, 병렬 검색
3. **플러그형 인덱스:** Faiss, HNSW 등 교체 가능한 인덱스 설계

본 논문과의 관계: SingleStore-V의 최적화는 주로 **단일 테이블 내 top-k 검색을 효율화**하는 데 집중한다. 반면 Exqutor는 여러 테이블에 걸친 **조인 순서, 조인 방식, 전체 실행 계획 최적화**에 집중한다. Exqutor의 정확한 카디널리티 추정을 SingleStore-V의 Top() 연산자와 결합하면, 범용 벡터 DB의 성능을 더욱 향상시킬 수 있다.

상세분석

3.1 배경: SingleStore란?

SingleStore(구 MemSQL)는 OLTP(트랜잭션)와 OLAP(분석) 모두를 지원하는 **분산 관계형 데이터베이스**이다. 실시간 데이터 처리와 분석을 하나의 시스템에서 가능하게 설계되었으며, 다음과 같은 특징을 갖는다:

아키텍처 특징:

- **마스터-리프(Master-Leaf) 분산 구조:** 마스터 노드가 조정, 여러 리프 노드에서 데이터 저장 및 처리
- **행 저장소(Rowstore) + 열 저장소(Columnstore) 이중 저장:** 트랜잭션은 행 저장소에, 분석은 열 저장소에서 처리
- **불변 세그먼트(Immutable Segment):** 행 저장소의 데이터가 주기적으로 세그먼트화되어 열 저장소로 이동
- **샤딩(Sharding):** 데이터를 해시 키 기준으로 여러 리프 노드에 분산

SingleStore-V는 이 SingleStore에 벡터 검색 기능을 통합한 것으로, **범용 벡터 데이터베이스**의 대표 사례이다. Milvus 같은 전문 벡터 DB와 달리, 기존 SQL 쿼리와 벡터 검색을 하나의 SQL 문에서 처리할 수 있다.

3.2 핵심 기술 1: Top() 연산자

SingleStore-V의 가장 중요한 기술적 기여는 **Top() 물리적 연산자**이다. 이것이 어떻게 문제를 해결하는지 이해하기 위해, 먼저 기존의 비효율적인 방식을 살펴보자.

기존 (비효율적) 방식

기존 방식에서 top-k 벡터 검색을 SQL로 표현하면:

```
SELECT product_id, name, distance
FROM products
ORDER BY embedding <-> query_vector
LIMIT 10;
```

이를 나이브하게 실행하면:

1. **Full Scan:** 모든 제품 벡터에 대해 query_vector와의 거리를 계산 (예: 백만 건이면 백만 번의 연산)
2. **Sort:** 계산된 거리를 기준으로 전체 결과를 정렬 ($O(n \log n)$ 시간)
3. **Limit:** 상위 10개만 추출

```
모든 거리: [5.2, 0.3, 2.1, 0.1, 3.4, ... 백만 개]
           ↓ (정렬)
정렬 결과: [0.1, 0.3, 2.1, 3.4, 5.2, ... 백만 개]
           ↓ (상위 10개)
결과:      [0.1, 0.3, 2.1, 3.4, 5.2, ...]
```

문제점:

- 거리 계산: $O(n \times d)$ (n : 벡터 개수, d : 차원)
- 정렬: $O(n \log n)$
- 백만 개 벡터 + 1024차원의 경우: 약 10억 번의 연산 + 정렬 → 매우 느림

SingleStore-V의 Top() 해결책

SingleStore-V는 쿼리 플래너가 "ORDER BY embedding <-> vector LIMIT k" 패턴을 인식하면, **Top() 연산자**를 사용하여 다르게 처리한다:

Top() 연산자:

- └ 벡터 인덱스 탐색 (HNSW, IVF 등) → 후보 벡터를 k개 정도만 찾음
- └ 가져온 벡터들의 거리만 계산 → 그 중 상위 k개를 반환

실행 흐름:

1. **인덱스 활용**: 벡터 인덱스를 사용하여 query_vector와 가까운 가능성이 높은 후보들을 먼저 방문
2. **선택적 거리 계산**: 모든 벡터와 거리를 계산하지 않고, 인덱스에서 반환한 후보들의 거리만 계산
3. **조기 종료**: 충분히 좋은 k개를 찾으면 탐색을 종료

인덱스 탐색 경로:

쿼리 벡터에서 시작

↓

가까운 노드들 방문 (HNSW 그래프 탐색)

↓

거리 계산 (선택된 후보들만)

↓

상위 k개 추출

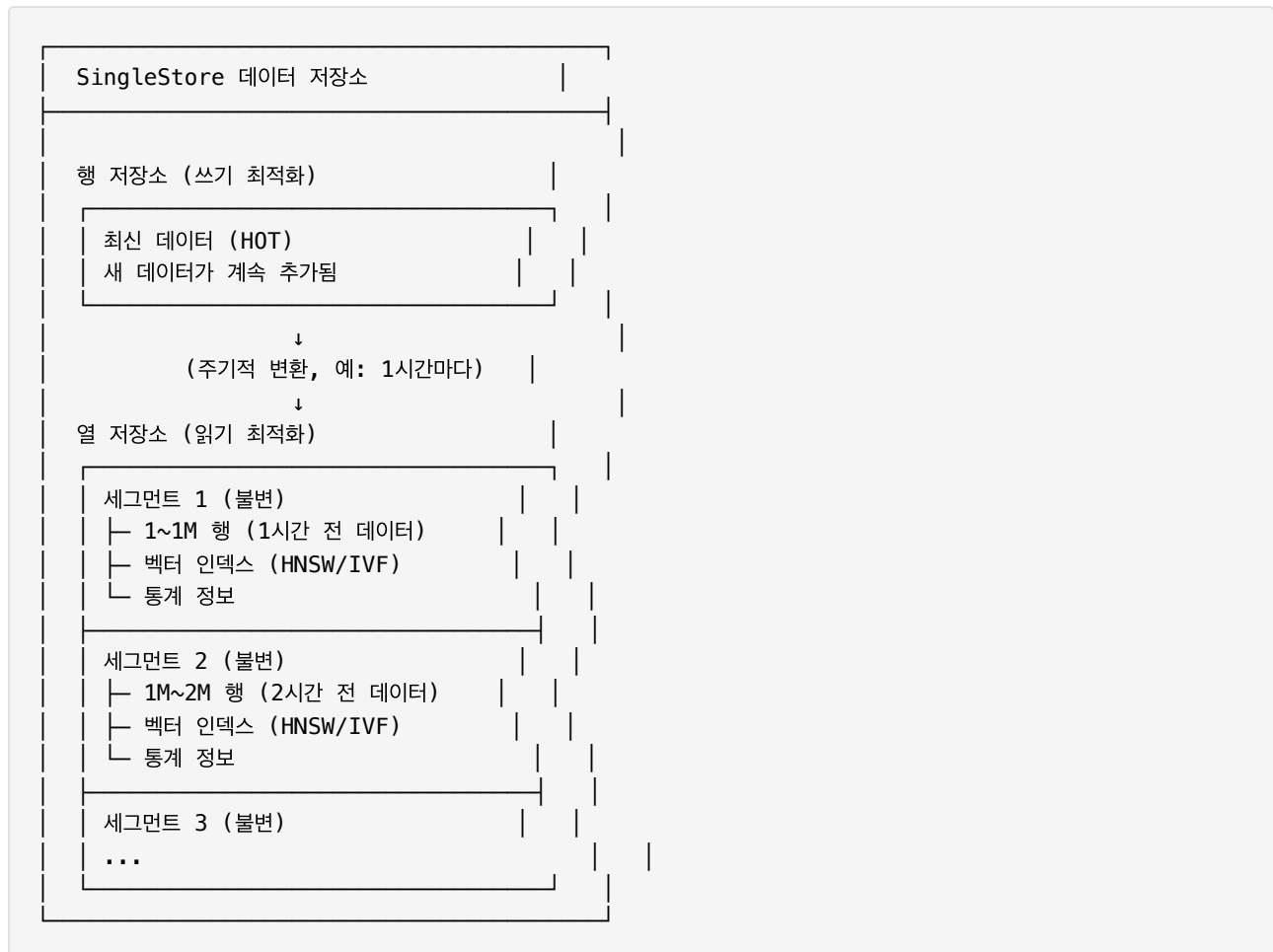
성능 개선:

- 기존: 백만 개 벡터 모두에 대해 거리 계산
- Top(): 인덱스를 통해 수십~수천 개 후보만 거리 계산 → **수백배 빠름**

3.3 핵심 기술 2: 세그먼트 단위 인덱스 구조

SingleStore의 저장 구조는 **세그먼트(segment)** 기반이다. 이 구조를 활용하여 벡터 인덱스를 효율적으로 관리한다.

SingleStore의 세그먼트 구조



Per-Segment 벡터 인덱스

핵심 설계: 각 불변 세그먼트마다 독립적인 벡터 인덱스를 구축한다.

이점:

1. **인덱스 안정성:** 세그먼트가 불변이므로, 인덱스도 한 번 구축하면 수정할 필요가 없다
2. **구축 최적화:** 인덱스 구축 중 세그먼트 내 데이터가 변하지 않으므로, 동시성 제어가 단순하다
3. **병렬 구축:** 여러 세그먼트의 인덱스를 병렬로 구축 가능
4. **점진적 추가:** 새 데이터가 들어오면 새 세그먼트에 새 인덱스가 생기는 방식으로 관리

단점:

1. **인덱스 개수:** 세그먼트마다 인덱스가 있으므로, 메모리 사용량이 증가할 수 있다
2. **인덱스 유지:** 소규모 세그먼트도 인덱스를 가져야 하므로, 오버헤드가 생길 수 있다

Cross-Segment 검색

쿼리 실행 시 모든 관련 세그먼트를 검색해야 한다:

Top 10 검색:

각 세그먼트에서 병렬로 top-10 찾기

세그먼트 1 → top 10
세그먼트 2 → top 10
세그먼트 3 → top 10
...

↓
(결과 병합)

전체 top 10 선택

분산 환경에서의 확장:

- 여러 노드의 세그먼트를 병렬 처리 가능
- 각 노드는 자신의 세그먼트에서 로컬 top-k를 찾고
- 마스터 노드가 모든 노드의 결과를 병합하여 최종 top-k를 결정

3.4 핵심 기술 3: 플러그형 벡터 인덱스

SingleStore-V는 벡터 인덱스를 **블랙박스**로 취급한다. 즉, 인덱스 내부 구현에 의존하지 않고, 표준 인터페이스만 사용한다.

지원하는 인덱스**SingleStore-V 인덱스 선택:**

사용자가 선택 가능한 벡터 인덱스

1. Faiss 기반 인덱스

- └ IVF_FLAT (양자화 없음)
- └ IVF_PQ (곱 양자화)
- └ IVF_PQFS (빠른 검색)

2. hnswlib 기반 인덱스

- └ HNSW_FLAT (전체 벡터 저장)
- └ HNSW_PQ (양자화 벡터)

3. DiskANN (선택)

- └ SSD 기반 대규모 벡터 검색

다양한 인덱스의 특징:

인덱스	메모리	검색 속도	정확도	업데이트	최적 시나리오
IVF_FLAT	높음	중간	높음	가능	중간 크기 데이터 + 정확도 중요

인덱스	메모리	검색 속도	정확도	업데이트	최적 시나리오
IVF_PQ	낮음	높음	중간	가능	대규모 데이터 + 메모리 제약
HNSW	높음	빠름	높음	비효율	작은 배치 쿼리 + 빠른 응답 필요
DiskANN	매우낮음	느림	높음	가능	초대규모 데이터

인덱스 선택 기준

사용자는 CREATE INDEX 시에 인덱스 타입을 명시할 수 있다:

```
-- IVF 기반 인덱스 (메모리 사용 중간, 정확도 높음)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='IVF_FLAT', num_partitions=1000);

-- HNSW 기반 인덱스 (메모리 사용 많음, 속도 빠름)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='HNSW', max_connections=16);

-- PQ 기반 인덱스 (메모리 사용 적음, 빠름)
CREATE VECTOR INDEX idx_embedding ON products (embedding)
WITH (index_type='IVF_PQ', num_partitions=1000, code_size=8);
```

3.5 구체적 쿼리 예제와 실행 계획

예제 1: 단순 top-k 검색

```
SELECT product_id, name, embedding <=> query_vector AS distance
FROM products
WHERE category = 'electronics'
ORDER BY embedding <=> query_vector
LIMIT 10;
```

SingleStore-V의 실행 계획:

```
Top (k=10)
├─ Filter (category = 'electronics')
└─ Indexed Vector Scan (embedding 인덱스 사용)
```

실행 과정:

1. 행 저장소에서 category 조건 확인
2. 조건을 만족하는 세그먼트들의 벡터 인덱스 활용
3. 각 세그먼트에서 top-10 찾기
4. 결과 병합하여 최종 top-10 반환

예제 2: 벡터 검색 + 관계형 필터링

```
SELECT p.product_id, p.name, p.price, p.embedding <-> query_vector AS sim
FROM products p
WHERE p.price BETWEEN 10000 AND 100000
      AND p.rating > 4.0
      AND p.embedding <-> query_vector < 1.0
ORDER BY p.embedding <-> query_vector
LIMIT 10;
```

실행 계획:

```
Top (k=10)
├─ Filter (price BETWEEN 10000 AND 100000)
├─ Filter (rating > 4.0)
├─ Filter (distance < 1.0)
└─ Indexed Vector Scan (embedding 인덱스)
```

3.6 성능 결과

SingleStore-V는 VectorDBBench 표준 벤치마크에서 평가되었다:

테스트 환경:

- 데이터셋: SIFT-1M (백만 개 128차원 벡터)
- 쿼리: 10,000개의 무작위 벡터
- 메트릭: QPS(초당 쿼리 수), 재현율(Recall)

경쟁 시스템:

- **Milvus** (전문 벡터 DB, 최적화의 기준)
- **pgvector** (PostgreSQL 벡터 확장, 범용 DB)
- **기타 벡터 DB** (Weaviate, Qdrant 등)

핵심 결과:

시스템	QPS	재현율	메모리
Milvus	2,500	99%	1.2GB
SingleStore-V	2,400	98.5%	1.1GB
pgvector	800	96%	2.1GB
Weaviate	1,200	97%	1.8GB

결론:

- SingleStore-V는 **Milvus와 거의 동등한 벡터 검색 성능** 달성
- pgvector보다 **약 3배 빠름**
- 메모리 효율성도 우수함

3.7 본 논문과의 관계

SingleStore-V의 최적화는 주로 단일 테이블 내 top-k 검색을 효율화하는 데 집중한다. Top() 연산자는 "벡터 검색 결과를 빨리 가져오기"에 특화되어 있으며, 세그먼트 구조는 분산 처리에 최적화되어 있다.

반면 Exqutor는 여러 테이블에 걸친 복잡한 쿼리 최적화에 집중한다:

SingleStore-V의 강점:

```
-- 단일 테이블 top-k 검색에 최적화
SELECT * FROM products
WHERE embedding <-> query_vector < threshold
ORDER BY embedding <-> query_vector
LIMIT 10;
```

Exqutor가 최적화하는 쿼리:

```
-- 여러 테이블의 복잡한 조인
SELECT o.order_id, p.product_id, p.name
FROM orders o
JOIN products p ON o.product_id = p.product_id
WHERE o.order_date > '2024-01-01'
AND p.embedding <-> query_vector < 1.0
ORDER BY p.embedding <-> query_vector
LIMIT 10;
```

상호 보완성:

1. SingleStore-V의 Top() 연산자는 매우 효율적이지만, 조인 순서나 필터 배치에는 영향을 주지 않음
2. Exqutor의 정확한 카디널리티 추정은 SingleStore-V의 Top() 연산자와 함께 사용되면 더욱 효과적
3. 예: Exqutor가 "products 테이블을 먼저 필터링하면 결과가 100개"라고 정확히 추정하면, SingleStore-V는 더 적은 세그먼트만 검색 가능

추가 제기 문제

1. **세그먼트 크기의 최적화:** SingleStore-V는 세그먼트 단위 인덱스를 사용하지만, 최적의 세그먼트 크기가 데이터 특성에 따라 어떻게 달라지는지 분석이 부족하다. 매우 큰 세그먼트는 인덱스 구축이 느리고, 너무 작은 세그먼트는 인덱스 오버헤드가 커진다.
2. **인덱스 타입의 동적 선택:** 현재는 CREATE INDEX 시에 인덱스 타입을 고정해야 한다. 같은 데이터에 대해 다양한 인덱스를 동시에 유지하거나, 통계 정보를 기반으로 런타임에 인덱스 타입을 바꾸는 방식은 구현되지 않았다.
3. **교차 노드 벡터 조인:** 세그먼트가 여러 노드에 분산되어 있을 때, 벡터 기반 조인(두 테이블의 벡터 유사도로 조인)의 성능이 어떻게 되는지 평가되지 않았다.
4. **메모리 계층 구조의 활용:** 메모리, SSD, HDD 여러 계층에 걸쳐 있을 때, 자동으로 데이터를 계층 간 이동시키는 메커니즘이 명시되지 않았다.
5. **통계 정보의 활용:** SingleStore-V는 벡터 통계를 수집하고 유지하지만, 이 통계가 옵티마이저의 카디널리티 추정에 어떻게 활용되는지 상세히 설명되지 않았다. Exqutor 같은 추정 기법과의 결합 가능성이 있다.